

第一题:

【第 13 章 数据存储结构】

a. MRU优于LRU的情况:

关系代数表达式: $R_1 \bowtie R_2$

查询处理策略: 采用嵌套循环连接 (Nested-Loop Join), 其中 R_2 中的每个元组需要与 R_1 的所有数据块进行比较。

- 示例:

- $R_1 = \{A, B, C\}$, $R_2 = \{1, 2\}$
- 每次处理 R_2 的一个元组 (如1) 时, 扫描 R_1 的每个块 (如A、B、C)。
- 在LRU策略下, 最近最少使用的数据块 (如A) 可能被替换, 导致重复加载。
- MRU策略则保留最近使用的块, 避免不必要的I/O操作。

b. LRU优于MRU的情况:

关系代数表达式: $R_1 \bowtie R_2$

查询处理策略: 采用排序连接 (Sort-Merge Join), 其中两个关系根据连接值排序并逐一比较。

- 示例:

- $R_1 = \{1, 3, 5, 7\}$, $R_2 = \{3, 4, 5, 6\}$
- 排序后按顺序扫描, 但由于重复连接值 (如5), 可能需要“回溯”到之前的数据块。
- MRU策略会替换最近使用的块, 导致回溯时重新加载块。
- LRU策略保留更久未使用的块, 更适用于这种情况。

第二题、第三题:

2、用下面的码值集合建立一棵 B+ 树:

(2, 3, 5, 7, 11, 17, 19, 23, 29, 31)

假设树初始为空, 按升序添加这些值, 一个节点最多所能容纳的指针数量为 4 (n=4)。请首先分别计

算根节点、内部节点 (非根非叶)、叶节点分别能容纳的指针数量区间, 然后画出构造过程的每一步并

简单加以解释。

解:

根节点: 当树只有一层时, 能容纳的指针数量[0,4]; 当树不止一层时, 能容纳的指针数量区间为[2,4]

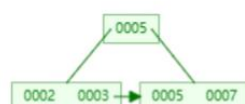
内部节点: [2,4]

叶节点: [3,4] (包含最少2个值最多3个值)

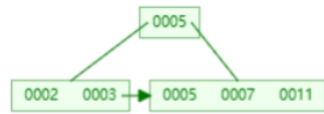
插入2, 3, 5:

0002 0003 0005

插入7: 由于叶子节点最多只能有三个, 插入7之后需要对该节点进行拆分, 拆分成两个分别含有两个值的节点, 并将拆分出来的新节点的最小搜索码值 (这道题中即5) 放入父节点当中, 如下:



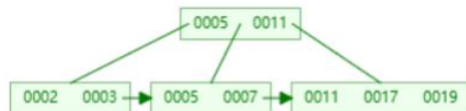
插入11：按照B+树插入的规则，直接将11放置在7的后面，此时符合B+树的特点，无需调整：



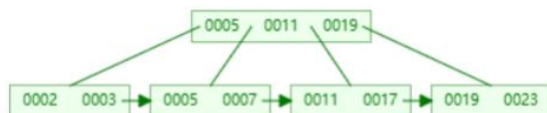
插入17：按照B+树的插入规则，17应该放置在11的后面，但此时叶子节点已经有四个值，超出了三个，所以需要对该叶子节点进行拆分，拆分成两个分别含有两个值的节点，并将拆分出来的新节点的最小搜索码值（即11）放入父节点当中，如下：



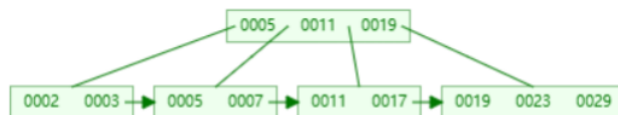
插入19：按照B+树插入的规则，直接将19放置在17的后面，此时符合B+树的特点，无需调整：



插入23：按照B+树的插入规则，23应该放置在19的后面，但此时叶子节点已经有四个值，超出了三个，所以需要对该叶子节点进行拆分，拆分成两个分别含有两个值的节点，并将拆分出来的新节点的最小搜索码值（即19）放入父节点当中，如下：



插入29：按照B+树插入的规则，直接将29放置在23的后面，此时符合B+树的特点，无需调整：



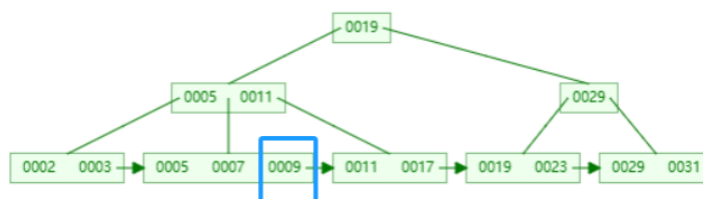
插入31：按照B+树的插入规则，31应该放置在29的后面，但此时叶子节点已经有四个值，超出了三个，所以需要对该叶子节点进行拆分，拆分成两个分别含有两个值的节点，并将拆分出来的新节点的最小搜索码值（即19）放入父节点当中，观察到，此时的父节点也已经四个值，超出了三个，同理，需要对父节点进行拆分，拆分成两个分别含两个值的两个节点，并将拆分出来的新节点的最小搜索码值（即19）放入到父节点当中，如下：



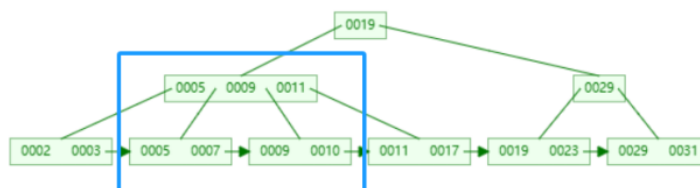
3、对于上一题中构建的B+树，依次执行以下操作，给出各操作后树的形状，并加以解释：

插入9、插入10、插入8、删除23、删除19。

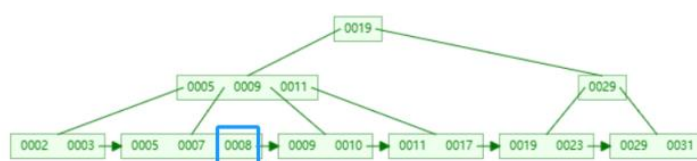
- 插入9：按照B+树的插入规则，9应该放置在第二个叶子节点中最为最后一个值，如下：



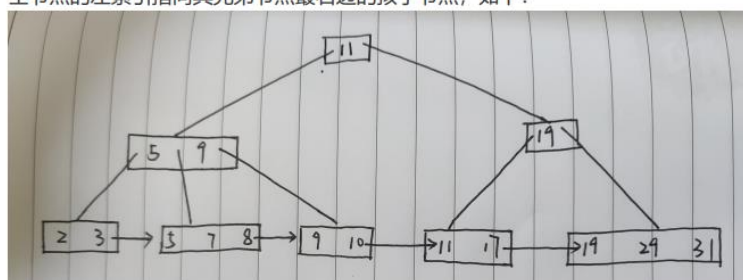
- 插入10：按照B+树的插入规则，10应该放置在第二个叶子节点中最为最后一个值，但此时该叶子节点已经含有四个值（超出三个），所以要进行拆分，拆分成两个分别含有两个值的节点，并将9放置到父节点当中，如下：



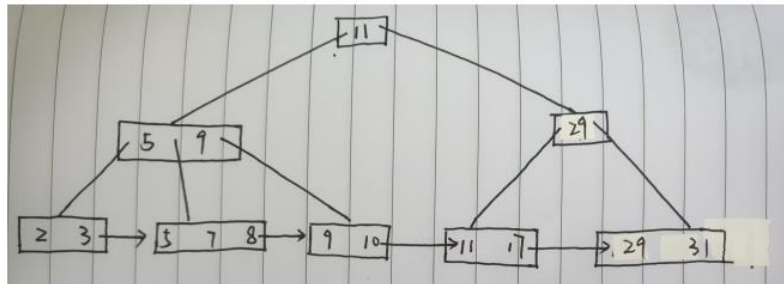
- 插入8：按照B+树的插入规则，8应该放置在第二个叶子节点中最为最后一个值，此时无需调整：



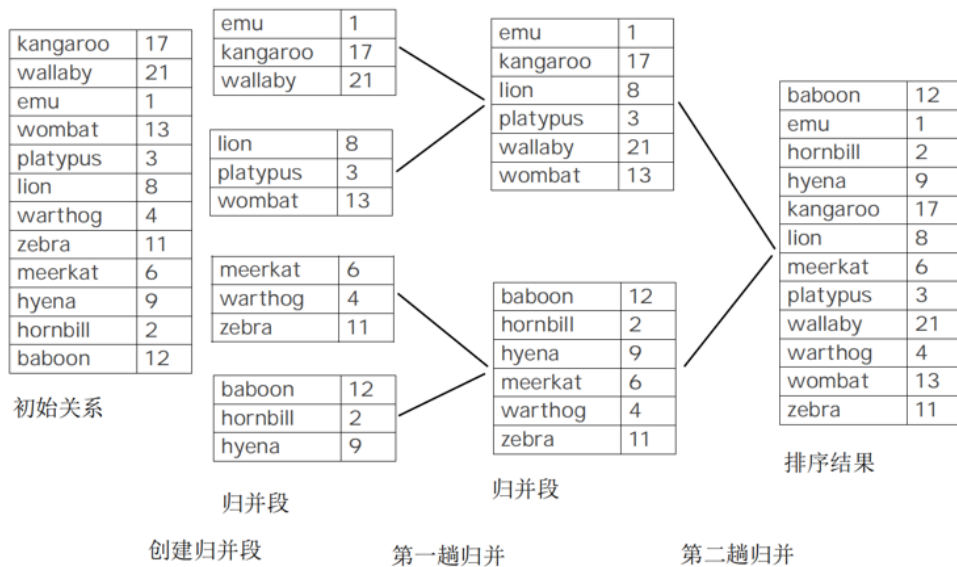
- 删除23：删除23之后，原本23所在的节点只剩下一个值，不符合B+树的平衡条件，该节点与其右兄弟节点合并，合并之后可得其父亲节点只剩下一个孩子，不符合B+树的平衡条件，所以将其所在的层数重新分配，将11提到根节点的位置，太空节点的左索引指向其兄弟节点最右边的孩子节点，如下：



- 删除19: 删除19之后, 符合平衡条件, 此时只需修改对应的父节点值为29, 如下:



第四题:



第五题:

解: 设关系 $r_1(A, B, C)$ 和 $r_2(C, D, E)$ 具有如下性质: r_1 有 20000 个元组, r_2 有 45000 个元组, 一个块中可容纳 25 个 r_1 元组或 30 个 r_2 元组。使用不同连接策略执行 $r_1 \bowtie r_2$, 分别计算需要的块传输和寻道次数。

由题意可得 ——

- 对于关系 r_1 : 元组数 $n_1 = 20000$, 块数 $b_1 = 20000 / 25 = 800$
- 对于关系 r_2 : 元组数 $n_2 = 45000$, 块数 $b_2 = 45000 / 30 = 1500$

假设内存可容纳的块数为 M , 考虑不同情况:

① 当 $M > 800$ 时, 即内存大小此时足以一次完全放入关系 r_1 。

最好情况下, 外层 r , 内层 s , 则块传输: $b_s + b_r$, 寻道次数: 2

考虑最好情况, 其块传输数量为 $b_1 + b_2 = 800 + 1500 = 2300$, 寻道次数为 2。

② 当 $2 < M \leq 800$ 时, 以内存最多能容纳的规模作为外层的分块单位, 即一次读取外层关系的 $M - 2$ 个块:

a) 嵌套-循环连接

外层 r , 内层 s , 则块传输: $n_r \cdot b_s + b_r$, 寻道次数: $n_r + b_r$

- r_1 为外层关系, r_2 为内层关系
 - 块传输: $n_1 \cdot b_2 + b_1 = 20000 \times 1500 + 800 = 30000800$
 - 寻道次数: $n_1 + b_1 = 20000 + 800 = 20800$
- r_2 为外层关系, r_1 为内层关系
 - 块传输: $n_2 \cdot b_1 + b_2 = 45000 \times 800 + 1500 = 36001500$
 - 寻道次数: $n_2 + b_2 = 45000 + 1500 = 46500$

b) 块嵌套-循环连接

外层 r , 内层 s , 则块传输: $\lceil \frac{b_r}{M-2} \rceil \cdot b_s + b_r$, 寻道次数: $2 \lceil \frac{b_r}{M-2} \rceil$

- r_1 为外层关系, r_2 为内层关系
 - 块传输: $\lceil \frac{b_1}{M-2} \rceil \cdot b_2 + b_1 = \lceil \frac{800}{M-2} \rceil \times 1500 + 800$
 - 寻道次数: $2 \lceil \frac{b_1}{M-2} \rceil = 2 \times \lceil \frac{800}{M-2} \rceil$
- r_2 为外层关系, r_1 为内层关系
 - 块传输: $\lceil \frac{b_2}{M-2} \rceil \cdot b_1 + b_2 = \lceil \frac{1500}{M-2} \rceil \times 800 + 1500$
 - 寻道次数: $2 \lceil \frac{b_2}{M-2} \rceil = 2 \times \lceil \frac{1500}{M-2} \rceil$

c) 归并-连接

1. 外排序 - 归并算法

一般化, 假设关系 r_1 和 r_2 并未按照连接属性排好序, 单个关系排序的代价为:

- 块传输: $b_i(2 \lceil \log_{\lfloor \frac{M}{b_b} \rfloor - 1}(\frac{b_i}{M}) \rceil + 1)$
- 寻道次数: $2 \lceil \frac{b_i}{M} \rceil + \lceil \frac{b_i}{b_b} \rceil (2 \lceil \log_{\lfloor \frac{M}{b_b} \rfloor - 1}(\frac{b_i}{M}) \rceil - 1)$

考虑最坏情况, 即为每个归并段分配 $b_b = 1$ 个缓冲块, 则对关系 r_1 、 r_2 进行归并外排序的代价为

- 块传输: $T_t = 800(2 \lceil \log_{M-1}(\frac{800}{M}) \rceil + 1) + 1500(2 \lceil \log_{M-1}(\frac{1500}{M}) \rceil + 1)$
- 寻道次数: $T_s = 2 \lceil \frac{800}{M} \rceil + 800(2 \lceil \log_{M-1}(\frac{800}{M}) \rceil - 1) + 2 \lceil \frac{1500}{M} \rceil + 1500(2 \lceil \log_{M-1}(\frac{1500}{M}) \rceil - 1)$

2. 归并 - 连接算法

假设内存能容纳在连接属性上具有相同值的所有元组构成的每个集合 S_s , 且为每个关系分配 b_b 个缓冲块。则该算法的代价为:

- 块传输: $b_1 + b_2 = 800 + 1500 = 2300$
- 寻道次数: $\lceil \frac{b_1}{b_b} \rceil + \lceil \frac{b_2}{b_b} \rceil = \lceil \frac{800}{b_b} \rceil + \lceil \frac{1500}{b_b} \rceil$

因此, 归并-连接策略的代价为: 块传输 $T_t + 2300$, 寻道次数 $T_s + \lceil \frac{800}{b_b} \rceil + \lceil \frac{1500}{b_b} \rceil$.

d) 散列-连接

因为 $b_1 = 800 < b_2 = 1500$, 说明 r_1 较小, 则选择关系 r_1 作为构造关系, r_2 作为探查关系。

只考虑不需要递归分区的情况。使用散列函数 h 将公共属性的值映射到 $\{0, 1, \dots, n_h\}$, 因为构造关系 r_1 的规模为 $b_1 = 800$, 要满足分区 r_i 不会发生散列表溢出, 即每个 n_h 分区的规模都小于等于 M , 则 $n_h \geq \frac{b_1}{M}$ 。

为了简单起见, 我们忽略散列索引所需空间, 令 $n_h = \frac{800}{M}$ 。

令 b_b 表示为输入和每个输出缓冲分配的内存块, 则散列-连接策略的代价为:

- 块传输: $3(b_1 + b_2) = 3 \times (800 + 1500) = 6900$
- 寻道次数: $2(\lceil \frac{b_1}{b_b} \rceil + \lceil \frac{b_2}{b_b} \rceil) + 2n_h = 2(\lceil \frac{800}{b_b} \rceil + \lceil \frac{1500}{b_b} \rceil) + \frac{1600}{M}$

第六题

b. (i) 共有两种串行执行

i) $T_{13} \Rightarrow T_{14}$

执行 T_{13} : $\text{read}(A); \text{read}(B); \Rightarrow A=0, B=0$

if $A=0$ then $B := B+1; \Rightarrow A=0, B=1$

$\text{write}(B); \text{mem}(A)=0; \text{mem}(B)=1$

再执行 T_{14} :

$\text{read}(B); \text{read}(A); \Rightarrow A=0, B=1$

if $B=0$ then $A := A+1 \Rightarrow A=0, B=1$ ($B \neq 0$)

$\text{write}(A); \text{mem}(A)=0, \text{mem}(B)=1$.

此时满足 $A=0 \vee B=0$.

ii) $T_{14} > T_{13}$: 显然 T_{13}, T_{14} 具有对称性, 故同上可得最终有:

$\text{mem}(A)=1, \text{mem}(B)=0$, 则满足 $A=0 \vee B=0$

即两事务的每一种串行执行都保持了数据库的一致性。

(1) 产生不可串行化的并发执行:

T_{13} read(A) read(B) if $A=0$ then $B:=B+1$ write(B)	T_{14} read(B) read(A) if $B=0$ then $A:=A+1$ write(A)
--	--

此时不可串行化。因 T_{13} 的 write(B) 不能与 T_{14} 的 read(B) 产生非冲突的交换; T_{13} 的 read(B) 与 T_{14} 的 write(A) 冲突。

(2) 对 T_{13} 加锁:

T_{13} :

lock-S(A)

lock-X(B)

read(A)

read(B)

if $A=0$ then $B:=B+1$

write(B)

unlock(B)

unlock(A)

对 T_{14} 加锁

T_{14} :

lock-X(A)

lock-S(B)

read(B)

read(A)

if $B=0$ then $A:=A+1$

write(A)

unlock(B)

unlock(A)

显然,上述加锁符合两阶段封锁协议。

会有调度方案导致死锁,若把上述^{事务} T_{14} 中加锁顺序更换,若如下并发:

T_{13} lock-S(A) lock-X(B) :	T_{14} lock-S(B) lock-X(A) :	← Stop here.
---	---	---------------------

发现执行会在标明处陷入死锁,因下一步无事务可以继续加锁从而执行事务。

19.21 Consider the log in Figure 19.5. Suppose there is a crash just before the log record $\langle T_0 \text{ abort} \rangle$ is written out. Explain what would happen during recovery.

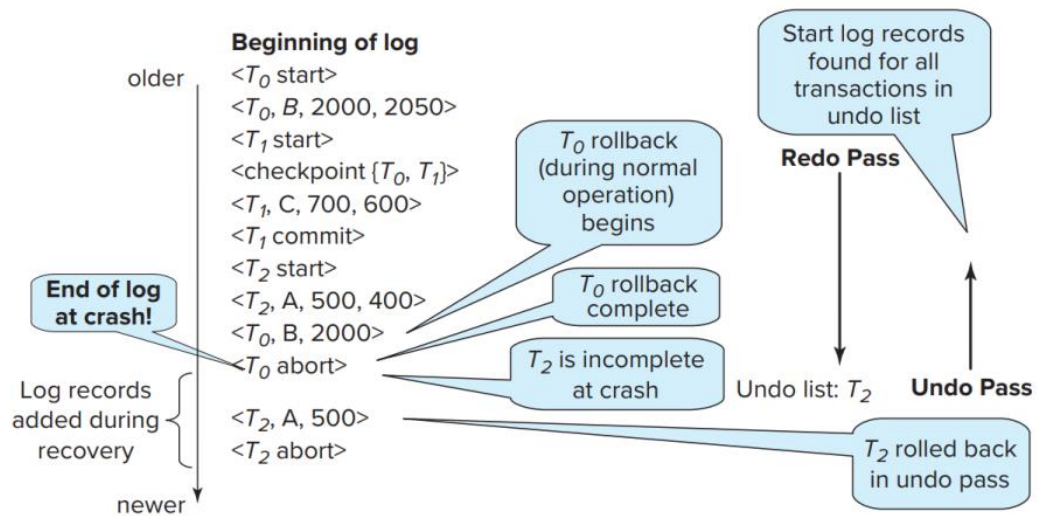


Figure 19.5 Example of logged actions and actions during recovery.

解：假设恰好在 $\langle T_0 \text{ abort} \rangle$ 日志记录被写出之前系统崩溃，分析在系统恢复时会发生的情况如下——

对于已被写出的日志记录：


```

< T0 start >
< T0, B, 2000, 2050 >
< T1 start >
< checkpoint{T0, T1} >
< T1, C, 700, 600 >
< T1 commit >
< T2 start >
< T2, A, 500, 400 >
< T0, B, 2000 >

```

系统崩溃后当数据库系统重启恢复，分两阶段进行：

1. 重做阶段 redo

系统从最后一个检查点日志记录 `<checkpoint{T0, T1}>` 开始正向扫描日志：

```

< checkpoint{T0, T1} > // 初始化回滚undo-list = {T0, T1}
< T1, C, 700, 600 > // 重做：C = 600
< T1 commit > // 去掉T1, undo-list = {T0}
< T2 start > // 加入T2, undo-list = {T0, T2}
< T2, A, 500, 400 > // 重做：A = 400
< T0, B, 2000 > // 重做：B = 2000

```

2. 撤销阶段 undo

由 1 可得，此时 `undo-list` 包括了在崩溃前尚未完成的所有事务，即 `undo-list = {T0, T2}`

•

我们从日志尾端开始反向扫描来执行回滚：

```

< T0, B, 2000 > // 为补偿日志记录，崩溃恢复时不会被执行undo
< T2, A, 500, 400 > // 撤销：A = 500，加入补偿日志记录 < T2, A, 500 >
< T2 start > // 去掉T2, undo-list = {T0}，且写入日志记录 < T2 abort >
< T1 commit > // T1 ∉ undo-list，不执行
< T1, C, 700, 600 >
< checkpoint{T0, T1} >
< T1 start >
< T0, B, 2000, 2050 > // 撤销：B = 2000，加入补偿日志记录 < T0, B, 2000 >
< T0 start > // 去掉T0, undo-list = {}，且写入日志记录 < T0 abort >

```

此时，`undo-list` 变为空表，撤销阶段结束。

由此可得，最后 $A = 500$ ， $B = 2000$ ， $C = 600$ ，且日志新写入 4 条记录，按顺序分别为：

```

< T2, A, 500 >
< T2 abort >
< T0, B, 2000 >
< T0 abort >

```